프로그래밍 입문

개발자의 첫 걸음

Git

GitHub

VS Code

버전 관리 · 원격 협업 · 개발 환경

bash

```bash
$ git init

$ git add .

$ git commit -m "first commit"

$ git push origin main

$ git pull
```

이 발표를 마치고 나면

- ✓ Git으로 코드 변경 이력을 관리할 수 있다
- ✓ GitHub로 코드를 공유·협업할 수 있다
- ✓ VS Code로 편리하게 개발할 수 있다

목 차

01 **왜 이 도구가 필요한가?**
파일 관리의 문제점과 해결 방향

02 **Git이란?**
분산 버전 관리 시스템 개념

03 **Git의 동작 구조**
3단계 작업 영역 + commit · branch

04 **GitHub란?**
원격 저장소 · push · pull · PR

05 **VS Code란?**
코드 에디터 + Git 내장 환경

06 **전체 흐름 정리**
VS Code → Git → GitHub 통합 워크플로

07 **실전 핵심 명령어**
add · commit · push 3단계 흐름

08 **마무리**
핵심 요약 및 다음 단계

왜 이 도구가 필요한가?

파일 관리의 현실적인 문제

현실에서 흔히 볼 수 있는 파일 목록

- 파일_v1.py
- 파일_최종.py
- 파일_최종2.py
- 파일_진짜최종.py
- 파일_진짜진짜최종.py

되돌리기 불가

수정 전 상태로
되돌아가기 어렵다

변경 추적 불가

무엇이 바뀌었는지
파악할 수 없다

협업 불가

여러 명이 같은 파일을
동시에 수정하면 충돌

해결책: Git + GitHub + VS Code 를 함께 사용한다

이제 각 도구가 어떻게 이 문제를 해결하는지 살펴보겠습니다.

Git이란?

Version Control System — 분산 버전 관리 시스템

버전 관리(Version Control)란?

파일이 언제, 어떻게 변경되었는지를 기록하여 언제든지 과거 상태로 되돌릴 수 있게 해주는 시스템

버전 관리

파일의 변경 이력을 체계적으로 저장

- 언제 바뀌었는가?
- 무엇이 바뀌었는가?
- 누가 바꾸었는가?

분산(Distributed)

각 사용자의 컴퓨터에 전체 기록 보유

- 인터넷 없이도 작업 가능
- 서버 장애가 나도 안전
- 빠른 속도로 이력 조회

핵심 가치

코드의 타임머신

- 언제든 과거로 되돌아갈 수 있다
- 실수를 두려워하지 않아도 된다
- 변경 이유를 기록으로 남긴다

한 줄 요약: Git = '코드의 타임머신'

Git의 동작 구조

3가지 작업 영역 + commit + branch

Working Directory

실제 파일을
수정하는 공간

git add →

→

Staging Area

커밋할 파일을
선택하는 공간

git commit →

→

Repository

최종 기록이
저장되는 공간

커밋 이력 보관

git add <파일>

git commit -m '메시지'

📸 commit이란?

특정 시점의 전체 파일 상태를 저장하는 스냅샷

- 단순 저장이 아니라 그 순간의 상태를 그대로 기록
- 언제든 해당 시점으로 복원 가능
- 메시지를 남겨 변경 이유를 기록할 수 있음

🌿 branch란?

기존 코드를 건드리지 않고 독립적으로 개발

- 새 기능을 안전하게 실험할 수 있음
- 여러 작업을 동시에 진행 가능
- 완성 후 원본(main)에 merge(병합)

GitHub란?

Git 저장소를 온라인에서 관리하고 공유하는 플랫폼

비유로 이해하기

Git이 '내 컴퓨터의 작업실'이라면, GitHub는 그 작업물을 올려두는 '클라우드 공유 공간'입니다.

내 컴퓨터 (Local)

GitHub (Remote)

git push → 업로드

← git pull 다운로드

원격 저장소

- 코드를 인터넷에 안전하게 백업
- 어디서든 접근 가능
- 분실·삭제 걱정 없음

Pull Request (PR)

- 코드를 바로 합치지 않고 팀원의 검토를 먼저 받음
- 코드 품질 유지의 핵심

Issue

- 버그 리포트 / 기능 제안 등록
- 담당자·라벨 설정 가능
- PR과 연결하여 자동 Close

한 줄 요약: GitHub = '개발자들의 구글 드라이브 + 코드 리뷰실'

VS Code란?

Microsoft가 만든 무료 오픈소스 코드 에디터

에디터(Editor)란?

코드를 작성하는 프로그램 — VS Code는 가벼우면서도 확장을 통해 강력한 IDE 수준의 기능을 제공합니다.

⚡ 가볍고 빠름

- 설치 용량 작고 실행 빠름
- 저사양 환경에서도 원활
- 즉시 사용 가능

확장(Extension)

- 30,000+ 플러그인
- Python·JS 등 모든 언어 지원
- Marketplace에서 1클릭 설치

Git 내장 연동

- GUI로 add·commit·push
- 변경사항 시각적 확인
- 터미널 없이도 Git 사용 가능

통합 터미널

- Ctrl+` 로 터미널 열기
- 코드 작성 → 실행 한 화면에
- 여러 터미널 동시 사용

스마트 자동완성

- IntelliSense 코드 자동완성
- 문법 오류 즉시 표시
- 함수 설명 팝업 지원

유용한 단축키

- Ctrl+P: 파일 빠른 열기
- Ctrl+Shift+F: 전체 검색
- Ctrl+/: 주석 토글

한 줄 요약: VS Code = '개발자의 만능 작업실'

전체 흐름 정리

VS Code → Git → GitHub 통합 워크플로

VS Code	+	Git	+	GitHub
코드 작성		**버전 관리**		**협업·공유**
개발 환경		로컬 이력		원격 저장소

개발 사이클

1	→	2	→	3	→	4	→	5	→	6
VS Code 코드 작성		git add 파일 선택		git commit 스냅샷 저장		git push GitHub 업로드		Pull Request 코드 리뷰		Merge 브랜치 병합

↺ 기능 하나당 새 브랜치 → 작업 → commit → push → PR → Merge 반복

VS Code 는 '코드 작성' | Git 은 '버전 관리' | GitHub 는 '협업' 을 담당합니다

실전 핵심 명령어

add · commit · push — Git 사용의 3단계 핵심 흐름

STEP 1

git add

변경된 파일 선택
(Staging Area로 이동)

```
git add 파일명.py    # 특정 파일만
git add .            # 모든 변경 파일
```

→

STEP 2

git commit

선택한 파일을 로컬에 저장
(스냅샷 기록)

```
git commit -m "메시지"  # 저장 + 설명 기록
git log                 # 커밋 이력 확인
```

→

STEP 3

git push

로컬 기록을 GitHub에 업로드
(팀원과 공유)

```
git push origin main  # 원격에 업로드
git pull              # 원격에서 내려받기
```

추가로 알아두면 좋은 명령어

git branch <이름>	새 브랜치 생성
git checkout <브랜치>	브랜치 이동
git merge <브랜치>	브랜치 병합

마무리

오늘 배운 핵심 내용 요약

Git

파일의 변경 이력을 로컬에서 관리하는 버전 관리 시스템
commit으로 스냅샷을 저장하고, branch로 독립적으로 작업한다

GitHub

Git 저장소를 클라우드에 올려 팀과 공유·협업하는 플랫폼
push·pull로 동기화하고, Pull Request로 코드 리뷰를 진행한다

VS Code

코드를 작성하는 가볍고 강력한 편집기
확장 설치, 통합 터미널, Git UI를 통해 개발 환경을 완성한다

핵심 3단계: git add → git commit → git push

📌 다음 단계 추천

① Pro Git Book (무료 한국어) — https://git-scm.com/book/ko
② GitHub Skills (실습 튜토리얼) — https://skills.github.com
③ VS Code 공식 튜토리얼 — https://code.visualstudio.com/docs

감사합니다! 🎉 질문 있으신가요?